Sentinel — Complete Project Package

ET AI Hackathon 2026 — PS7: Cyber Resilience for Critical National Infrastructure

This document combines the full hackathon documentation, the Software Requirements Specification, the system architecture, the Google Antigravity Master Build Prompt, and a rules/judging-criteria compliance pass into a single reference.

Contents:

Project Sentinel

Autonomous Cyber Defense for Critical Infrastructure

ET AI Hackathon 2026 — Problem Statement 7: Cyber Resilience for Critical National Infrastructure

---

1. Executive Summary

Critical infrastructure — power grids, water treatment plants, hospital networks — runs on legacy systems that were never designed for modern cyber threats. Human security teams are overwhelmed by alert volume and routinely miss real attacks in the noise.

Sentinel is a five-agent AI system that acts as a 24/7 security operations team: it watches network activity continuously, investigates anomalies using real threat-intelligence data, decides with a calibrated confidence score whether something is a genuine attack, takes only pre-approved safe actions, and records every decision in a tamper-evident audit log.

One-line pitch: An AI agent team that watches over power grids, water systems, and hospital networks 24/7, catches cyberattacks in seconds, takes safe pre-approved actions automatically, and keeps a tamper-evident record of every decision it makes.

---

2. Problem Statement

Modern critical infrastructure faces three compounding challenges:

1. **Legacy systems, modern threats** — OT/ICS environments were built for reliability, not security, and are increasingly targeted by ransomware, state actors, and opportunistic attackers.
2. **Alert fatigue** — Security teams are flooded with alerts; the real attack is often buried under thousands of false positives.
3. **Accountability gap** — When automated systems do act, there's often no verifiable record of why, making regulatory compliance (NERC CIP, SOC 2, ISO 27001) difficult and eroding trust in automation.

Sentinel addresses all three: continuous monitoring, confidence-calibrated decision-making, and provable auditability.

---

3. System Architecture

3.1 Pipeline Overview

```
Input (network activity / suspicious event)
        ↓
1. WATCHER Agent      → constantly scans traffic, spots anything unusual (fast, no AI)
        ↓
2. INVESTIGATOR Agent  → AI + real threat databases dig into what was found
        ↓
3. JUDGE Agent         → AI decides: Normal / Suspicious / Confirmed Attack — with confidence score
        ↓
   Confidence low? → asks Investigator a sharper question (1 retry only)
   Still unsure?   → escalates to a human instead of guessing
        ↓
4. RESPONDER Agent     → picks ONE safe, pre-approved action — never freeform
        ↓
5. RECORD Agent        → writes every step into a tamper-evident, verifiable log
```

3.2 Agent Definitions

1. Watcher Agent (rule-based, no AI)
Continuously scans a live (simulated/replayed) stream of network activity for red flags: traffic spikes, repeated failed logins, known-bad IP addresses, unusual commands sent to control systems. Runs instantly on simple rules — no model inference — and flags candidates for deeper investigation.

2. Investigator Agent (AI + real external data)
Enriches the Watcher's flag using live third-party sources:

- **AbuseIPDB** — IP reputation checks
- **VirusTotal** — file/link/hash reputation across security engines
- **GreyNoise** — distinguishes targeted attacks from internet background noise

The AI synthesizes this evidence into a structured summary: what's happening, severity, and likely attack category (DDoS, brute-force, data exfiltration, ransomware staging, etc.).

3. Judge Agent (AI, confidence-calibrated decision)
Produces a confidence score and reasoning rather than a binary verdict:

- **>80% confidence** → treated as a real incident, action triggered
- **40–80% confidence** → flagged for human review, no automatic action
- **<40% confidence** → logged quietly, no alert raised

If confidence is ambiguous, the Judge can request **one bounded retry** — asking the Investigator a sharper follow-up question — before either deciding or escalating to a human. This retry-then-escalate loop is the system's core self-correction mechanism and its strongest technical differentiator.

4. Responder Agent (AI, constrained action space)
Selects **only** from a fixed, pre-approved menu — never a freeform action:

- Block the suspicious IP address
- Isolate the affected network segment
- Alert the human on-call engineer
- Open an incident ticket
- Trigger failover to backup system

Constraining the action space is both a safety mechanism and the system's core compliance story: an AI agent that can only choose from a human-approved menu cannot take an unexpected or dangerous action against real infrastructure.

5. Record Agent (deterministic, no AI)
Logs every step — detection, evidence, decision, confidence score, action taken, responsible agent — into a **tamper-evident hash chain**, where each record is cryptographically linked to the one before it. Any retroactive alteration breaks the chain and is immediately detectable on verification. This maps directly to real compliance frameworks (NERC CIP for power grids, SOC 2 / ISO 27001 generally).

> **Important framing note:** This is *tamper-evident and verifiable*, not cryptographically *immutable* — anyone with direct database write access could still rewrite the whole chain and recompute hashes. The pitch should use "tamper-evident" and "verifiable" language rather than claiming immutability, since security-literate judges will probe this distinction.

---

4. Why This Avoids False Alarms and Bad Actions

- **No single agent decides alone** — Investigator gathers facts, Judge decides, Responder acts. Three separate checks, not one black box.
- **Confidence scores, not blind alerts** — low-confidence findings are logged quietly rather than raising noise.
- **"Not sure" is a valid outcome** — the system can escalate to a human instead of guessing, avoiding both false alarms and missed attacks.
- **Constrained action space** — the Responder can never take an unapproved or unexpected action on real infrastructure.
- **Full provability** — every decision is logged and independently verifiable; nothing is a black box.

---

5. Tech Stack

Layer	Choice	Notes
Frontend	React + Tailwind	Dark, professional "control room" dashboard
Backend	Node.js + Express	Runs the 5-agent pipeline as a hand-rolled state machine
Database	MongoDB	Stores tamper-evident hash-chained log, case history, agent state
AI	Claude or Gemini API	Powers Investigator, Judge, Responder — structured JSON output
Threat intel	AbuseIPDB, VirusTotal, GreyNoise (free tiers)	Real, live, citable external data sources
Real-time updates	Socket.IO	Streams each agent's activity to the dashboard live

Orchestration note: A hand-rolled finite-state machine is used instead of a framework like LangGraph/CrewAI. This trades flexibility for reliability under hackathon time pressure and produces a more predictable, debuggable pipeline for a live demo.

Security-of-the-orchestrator note: For a security-pitched product, the pitch should include at least one sentence on how the orchestration layer itself is secured (auth/access control on the agent pipeline) — this is currently a gap and worth a line of coverage for credibility with security-literate judges.

---

6. Build Order (36–48 Hour Hackathon Timeline)

Time	Task
Hours 0–4	Team setup, repo, MongoDB schema, tamper-evident log logic
Hours 4–10	Watcher Agent (rule-based detection) + pipeline working end-to-end on dummy data
Hours 10–18	Investigator Agent — wire up real AbuseIPDB / VirusTotal / GreyNoise calls
Hours 18–26	Judge Agent — confidence scoring + retry/escalation logic (hardest part; start early, test hardest)
Hours 26–32	Responder Agent (constrained action menu) + dashboard live agent-graph animation
Hours 32–38	Audit log UI + "verify chain" button + end-to-end run-throughs
Hours 38–44	Bug fixes, prepare the "ambiguous case" demo moment, polish dashboard
Final hours	Pitch deck, impact numbers, video, rehearsal — no new code

Sequencing risk to manage: The Judge's retry/escalation logic (hours 18–26) is the highest-risk build and the most likely to overrun. The audit-log UI — the single best "wow" visual — is scheduled late (hours 32+). Consider building a thin/static version of the ledger UI in parallel earlier, then wiring it to real data once the pipeline stabilizes, so a pipeline overrun doesn't force a rushed version of the most demo-critical screen.

Demo resilience: Live calls to AbuseIPDB/VirusTotal/GreyNoise during the actual demo carry real risk (rate limits, lag). Cache a known-good set of real API responses as a fallback path so the live demo never depends simultaneously on live internet and third-party uptime.

---

7. What to Build for Real vs. What to Mock

Build for real:

- The full 5-agent pipeline, end-to-end
- Real API calls to AbuseIPDB + VirusTotal + GreyNoise (a handful of real lookups is sufficient)
- Confidence scoring + retry/escalation logic
- The tamper-evident, verifiable log
- The constrained action-space enforcement

Safe to mock/simplify:

- Network telemetry — use a pre-scripted, realistic replay of traffic/log data instead of a live network tap
- "Isolating a segment" / "failover" — log and simulate the action rather than integrating with real infrastructure control systems (never connect a hackathon demo to real infrastructure controls)
- Historical case-matching — seed a small set of pre-built prior cases rather than building a full similarity-search system

---

8. Demo Flow (3 Minutes)

Time	Beat
0:00–0:20	Open with real stakes: a hospital or power grid under attack, human team too slow/overwhelmed to catch it
0:20–1:00	Live demo: trigger a simulated attack (traffic burst from a known-bad IP hitting a control system) → watch all 5 agents activate on the live dashboard → Responder isolates the affected segment automatically, in under a minute
1:00–1:30	Show a deliberately ambiguous case (legitimate but unusual traffic spike) → system correctly returns "medium confidence, flagged for human review" instead of falsely blocking a real system. This is the moment that proves the false-alarm problem is solved — do not skip it
1:30–2:00	Show the tamper-evident audit log — click "verify chain" and watch it recompute and confirm nothing was altered, live
2:00–2:30	Show the constrained action menu and explain why the AI can never take an unapproved action on real infrastructure
2:30–3:00	Close with impact numbers: current average incident response time vs. Sentinel's response time, plus the compliance/audit-readiness angle

---

9. Compliance & Real-World Framing

Sentinel's design choices map directly onto real regulatory and audit needs:

- **Constrained action space** → demonstrable operational safety control, relevant to NERC CIP-style change-management requirements
- **Tamper-evident audit log** → verifiable incident record, relevant to SOC 2 / ISO 27001 audit trails
- **Confidence-gated escalation** → defensible human-in-the-loop control for high-stakes actions

This gives Sentinel a credible path from hackathon prototype to pilot-able product: B2B deployment alongside existing SOC tooling, with the audit log and constrained action space as the primary trust-building features for infrastructure operators.

---

10. Risks & Open Items

- **Immutability language** — always describe the log as "tamper-evident" and "verifiable," never "immutable," in the written pitch and live Q&A.
- **Third-party API dependency during demo** — mitigate with cached known-good responses as fallback.
- **Judge/retry logic build risk** — highest-effort component; start early, build the ledger UI in parallel rather than sequentially after it.
- **Orchestrator security** — add explicit coverage (even one sentence) of how the agent pipeline itself is authenticated/access-controlled.

---

11. Team Talking Points (Quick Reference)

- "Three independent checks before any action is taken — no single agent decides alone."
- "The system is allowed to say 'I'm not sure' — that's a feature, not a gap."
- "Every action comes from a fixed, pre-approved menu — this AI can never do something unexpected to real infrastructure."
- "Every decision is tamper-evident and independently verifiable — nothing is a black box."

PART 2: SOFTWARE REQUIREMENTS SPECIFICATION & BUILD PROMPT

Sentinel — SRS & Antigravity Master Build Package

---

1. Product Requirements Document (PRD)

Executive Summary

Sentinel is an AI agent pipeline that monitors network activity for critical infrastructure (power, water, hospitals), investigates anomalies against real threat-intelligence sources, and takes only pre-approved safe actions — logging every decision in a tamper-evident, verifiable chain. It replaces alert-fatigued human triage with a confidence-calibrated, auditable, five-agent system.

Core Workflows

- **Operator logs in** → lands on the live "control room" dashboard showing the current pipeline state and recent incidents.
- **Watcher flags an anomaly** → event appears in real time on the dashboard as "Investigating."
- **Investigator enriches the event** → operator can expand the card to see AbuseIPDB/VirusTotal/GreyNoise evidence as it streams in.
- **Judge scores confidence** → card updates to Low / Medium / High with a visible confidence score and reasoning; Medium-confidence events surface in a "Needs Review" queue.
- **Operator reviews an ambiguous case** → approves, dismisses, or requests more investigation (triggers one bounded Investigator retry).
- **Responder executes an approved action** → operator sees the exact pre-approved action taken (block IP / isolate segment / alert on-call / open ticket / trigger failover), never a freeform action.
- **Operator opens the Audit Log** → filters by date/severity/agent, clicks "Verify Chain" to re-hash and confirm no record has been altered.
- **Operator exports a compliance report** → generates a PDF/CSV summary of incidents and actions for a given period (NERC CIP / SOC 2 style).

Data Models (MongoDB / Mongoose)

```
Event
  _id
  source (watcher | manual)
  rawSignal: { ip, protocol, payloadSummary, timestamp }
  status: enum [flagged, investigating, judged, responded, closed]
  createdAt

Investigation
  _id
  eventId (ref Event)
  abuseIpdbResult: { score, categories, lastReportedAt }
  virusTotalResult: { maliciousCount, totalEngines, tags }
  greyNoiseResult: { classification, isTargeted }
  aiSummary: string
  attackCategoryGuess: enum [ddos, bruteforce, exfiltration, ransomware_staging, unknown]
  retryCount: number
  createdAt

Judgment
  _id
  eventId (ref Event)
  investigationId (ref Investigation)
  confidenceScore: number (0-100)
  verdict: enum [normal, suspicious, confirmed_attack]
  reasoning: string
  escalatedToHuman: boolean
  createdAt

Action
  _id
  eventId (ref Event)
  judgmentId (ref Judgment)
  actionType: enum [block_ip, isolate_segment, alert_oncall, open_ticket, trigger_failover]
  approvedBy: enum [system_auto, human_operator]
  status: enum [executed, simulated, rejected]
  createdAt

AuditRecord (tamper-evident chain)
  _id
  eventId (ref Event)
  sequenceNumber: number
  payload: { agent, action, evidence }
  previousHash: string
  currentHash: string
  createdAt

User
  _id
  name
  email
  role: enum [operator, admin]
  createdAt
```

Relationships: `Event 1—1 Investigation`, `Event 1—1 Judgment`, `Event 1—1 Action`, `Event 1—N AuditRecord` (one record per pipeline step). `AuditRecord.previousHash` always equals the prior record's `currentHash`, forming the chain per Event (and optionally chained globally across all events for a single verifiable ledger).

---

2. System Architecture Blueprint

Folder Structure

```
sentinel/
├── app/
│   ├── layout.tsx
│   ├── globals.css
│   ├── page.tsx                    # Landing / control room redirect
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── layout.tsx
│   ├── dashboard/
│   │   ├── page.tsx                # Live control room
│   │   ├── loading.tsx
│   │   └── components/
│   │       ├── PipelineGraph.tsx   # Live 5-agent visual, socket-driven
│   │       ├── EventCard.tsx
│   │       ├── ConfidenceBadge.tsx
│   │       └── ReviewQueue.tsx
│   ├── audit-log/
│   │   ├── page.tsx
│   │   └── components/
│   │       ├── AuditTable.tsx
│   │       └── VerifyChainButton.tsx
│   ├── reports/
│   │   └── page.tsx                # Compliance export UI
│   └── api/
│       ├── events/route.ts         # GET/POST events
│       ├── events/[id]/route.ts
│       ├── investigate/route.ts    # triggers Investigator Agent
│       ├── judge/route.ts          # triggers Judge Agent
│       ├── respond/route.ts        # triggers Responder Agent
│       ├── audit/route.ts          # GET log, POST verify-chain
│       └── reports/export/route.ts
├── components/
│   ├── ui/
│   │   ├── GlassPanel.tsx          # reusable glassmorphism container
│   │   ├── Button.tsx
│   │   ├── Badge.tsx
│   │   └── Sidebar.tsx
│   └── shared/
│       ├── Navbar.tsx
│       └── StatusDot.tsx
├── lib/
│   ├── agents/
│   │   ├── watcher.ts
│   │   ├── investigator.ts
│   │   ├── judge.ts
│   │   ├── responder.ts
│   │   └── record.ts
│   ├── integrations/
│   │   ├── abuseipdb.ts
│   │   ├── virustotal.ts
│   │   └── greynoise.ts
│   ├── db/
│   │   ├── mongoose.ts
│   │   └── models/
│   │       ├── Event.ts
│   │       ├── Investigation.ts
│   │       ├── Judgment.ts
│   │       ├── Action.ts
│   │       └── AuditRecord.ts
│   ├── hashChain.ts                # sha256 chaining + verify logic
│   ├── socket.ts                   # Socket.IO server bootstrap
│   └── auth.ts
├── types/
│   └── index.ts
├── .env.local
├── next.config.ts
├── tailwind.config.ts
├── package.json
└── tsconfig.json
```

Component Hierarchy (Dashboard)

```
DashboardPage
 └ GlassPanel (Pipeline Overview)
     └ PipelineGraph (Framer Motion node/edge animation, socket-fed)
 └ GlassPanel (Live Events)
     └ EventCard[]
         └ ConfidenceBadge
         └ EvidenceAccordion
 └ GlassPanel (Review Queue)
     └ ReviewQueue → ReviewItem[]
```

State Management & API

- **Server state**: fetched via Next.js Server Components for initial page load (events, audit log, reports) — no client-side loading spinners on first paint.
- **Live updates**: Socket.IO pushes agent-pipeline events to the dashboard as they happen (`event:flagged`, `event:investigating`, `event:judged`, `event:responded`). Client subscribes in a `useSentinelSocket()` hook and merges into local state via `useReducer`.
- **Mutations**: Server Actions for operator actions (approve/dismiss/request-retry) — no manual fetch boilerplate, direct form-action bindings from `ReviewItem`.
- **Agent triggers**: `/api/investigate`, `/api/judge`, `/api/respond` are internal route handlers invoked by the pipeline orchestrator (`lib/agents/*`), not called directly by the client.
- **Chain verification**: `/api/audit` with `?action=verify` recomputes the hash chain server-side and returns pass/fail + first broken index if any.
- **Caching**: Threat-intel API responses cached in Mongo with a TTL (e.g., 15 min) to protect demo reliability against live rate limits.

---

3. Google Antigravity Master Build Prompt

Copy everything in the block below into the Antigravity prompt box as a single `/goal`.

```
/goal

ROLE & CONTEXT
You are building "Sentinel" — a Next.js (App Router, TypeScript) application that visualizes
and runs a 5-agent AI cyber-defense pipeline for critical infrastructure. The agents are:
Watcher (rule-based detection), Investigator (AI + real threat-intel APIs), Judge (AI,
confidence-scored decisions with bounded retry), Responder (AI, constrained action menu only),
and Record (tamper-evident hash-chained audit log). This is a hackathon-grade but
production-styled build: it must run end-to-end, look premium, and be demoable live.

MANDATED STACK (do not substitute)
- Next.js App Router + TypeScript
- Tailwind CSS + Framer Motion + Lucide React
- MongoDB via Mongoose, accessed through Next.js API routes / Server Actions
- Socket.IO for live dashboard updates

DESIGN RULES (enforce on every screen, no exceptions)
- Dark theme only. Background: #0a0a0f (deep near-black), never pure black or default gray.
- Glassmorphism panels everywhere data is grouped: translucent background
  (bg-white/5 or similar), backdrop-blur-md, 1px border at low opacity (border-white/10),
  soft rounded corners (rounded-xl or larger).
- High-contrast text: near-white headings, muted gray-400 body text, no low-contrast gray-on-gray.
- Micro-animations via Framer Motion on: pipeline node activation, card entry/exit,
  confidence badge color transitions, chain-verify success/fail states. Keep them fast
  (150-300ms) and purposeful, never decorative-only.
- Icons via lucide-react only — no emoji, no other icon sets.
- No lorem-ipsum, no gray placeholder boxes, no "TODO: add content" — every screen must
  render with realistic, on-topic seed/mock data immediately after scaffold.
- Fully responsive: dashboard must degrade gracefully to a single-column stacked layout
  below 768px without losing the pipeline visualization (collapse to a vertical list).

STEP-BY-STEP TASK PLAN (execute phases sequentially, do not skip ahead)

PHASE 1 — Scaffold
1. Run: npx create-next-app@latest sentinel --typescript --tailwind --app --no-src-dir
2. Install: framer-motion, lucide-react, mongoose, socket.io, socket.io-client
3. Set up tailwind.config.ts with the dark palette as custom colors (background, panel, border, accent).
4. Create .env.local with placeholders: MONGODB_URI, ABUSEIPDB_KEY, VIRUSTOTAL_KEY, GREYNOISE_KEY.
   Do not commit real keys. Stop and ask me for keys if a real API call is required to proceed.

PHASE 2 — UI Shell
1. Build components/ui/GlassPanel.tsx as the single reusable glassmorphism container
   used everywhere. All future panels must use this component, not ad-hoc divs.
2. Build the Navbar and Sidebar with lucide-react icons for: Dashboard, Audit Log, Reports.
3. Build the dashboard route with a static-but-realistic mock version of PipelineGraph
   and EventCard, using seeded mock events. This must look complete and premium before
   any backend logic exists.

PHASE 3 — Database Layer
1. Implement lib/db/mongoose.ts connection singleton.
2. Implement all five Mongoose models exactly as specified in the SRS data model section
   (Event, Investigation, Judgment, Action, AuditRecord), plus User.
3. Implement lib/hashChain.ts: sha256-based chaining and a verifyChain(events) function
   that returns { valid: boolean, brokenAtIndex: number | null }.
4. Seed the database with 10-15 realistic mock events spanning all confidence bands.

PHASE 4 — Agent Logic
1. Implement lib/agents/watcher.ts as a rule-based scanner over the seeded/replayed event stream.
2. Implement lib/agents/investigator.ts: call AbuseIPDB, VirusTotal, GreyNoise (with a cached
   fallback response if any call fails or a key is missing), then call the LLM to synthesize
   a structured summary.
3. Implement lib/agents/judge.ts: LLM call returning strict JSON { confidenceScore, verdict,
   reasoning }. If confidence is ambiguous (roughly 40-60), trigger exactly one retry to the
   Investigator with a sharper follow-up question, then finalize or escalate.
4. Implement lib/agents/responder.ts: LLM call constrained to return ONLY one of the five
   allowed actionType enum values — validate the response server-side and reject/retry if
   the model returns anything outside the enum.
5. Implement lib/agents/record.ts: writes every step to AuditRecord via the hash chain.

PHASE 5 — Wiring (real-time + API)
1. Implement Socket.IO server bootstrap in lib/socket.ts and wire it into a custom server
   or route handler as appropriate for this Next.js version.
2. Implement /api/events, /api/investigate, /api/judge, /api/respond, /api/audit routes.
3. Wire the dashboard's PipelineGraph and EventCard components to live socket events,
   replacing the static mock data from Phase 2 without changing its visual design.
4. Implement the Review Queue with Server Actions for approve / dismiss / request-retry.
5. Implement the Audit Log page with the "Verify Chain" button calling /api/audit?action=verify,
   with an animated success/fail state via Framer Motion.

PHASE 6 — Verification Pass
1. Run the full pipeline end-to-end on at least 3 seeded scenarios: a clear attack (high
   confidence), an ambiguous case (medium confidence, triggers review queue), and background
   noise (low confidence, silently logged).
2. Confirm the hash chain verification correctly passes on an unmodified log and correctly
   fails (with the right broken index) if a record is manually edited in the database.
3. Confirm responsive layout at 375px, 768px, and 1440px widths.
4. Report back a summary of what was built, what was mocked vs. real, and any remaining
   TODOs before considering the build complete.

EXECUTION CONSTRAINTS
- Auto-run all safe, non-destructive commands (installs, file creation, dev server starts,
  local seed scripts) without asking for approval.
- STOP and ask for explicit approval before: deleting any file or directory, running any
  command that writes outside the project folder, dropping/resetting the database, or
  making any real external API call that could consume paid quota.
- Never fabricate or hardcode a real-looking API key. Use environment variable references only.
- After each phase, run the dev server and visually/functionally confirm the phase's
  deliverable works before starting the next phase.
- If a required package or command fails, stop and report the exact error rather than
  silently substituting a different library or approach.
```

---

4. Build-Prompt Gap Notes & Open Decisions

Included that's commonly missed:

- A dedicated `lib/integrations/` layer separating third-party API calls from agent logic, so the Investigator agent isn't tightly coupled to AbuseIPDB/VirusTotal directly.
- `lib/hashChain.ts` as its own module, separate from the Mongoose model — this is the piece most people bury inside a route handler and then can't test independently.
- A `PHASE 6 — Verification Pass` — most build prompts stop at "wire everything together" and never make the agent actually re-check its own output against defined test scenarios before declaring done.
- Explicit env var placeholders and a rule telling Antigravity to *stop and ask* rather than fabricate keys — without this, agentic build tools sometimes hardcode fake-looking credentials that silently fail later.
- A rule to visually confirm each phase before moving to the next — prevents the agent from building Phase 5 wiring on top of a broken Phase 3 database layer.

Deliberately left out — fill these in before running the build:

- **Auth provider specifics** — a `login/` route and `lib/auth.ts` are scaffolded, but NextAuth vs. Clerk vs. custom JWT isn't mandated. Decide before running Phase 1.
- **Deployment target** — nothing here specifies Vercel vs. self-hosted vs. Docker. Add a Phase 7 if deploying beyond a local demo.
- **Real vs. mock LLM provider** — the prompt says "call the LLM" generically. Decide Claude vs. Gemini vs. OpenAI and put the actual SDK/package name in Phase 1's install step.
- **Rate-limit handling detail** — a cached fallback for API failures is specified but not exact retry/backoff parameters. Fine for a demo; add specifics if this goes further.

PART 3: HACKATHON RULES & JUDGING CRITERIA COMPLIANCE

New section — reconciles Sentinel's design and execution plan against the official ET AI Hackathon 2026 rules and judging criteria.

12. Rules Compliance Checklist

Rule	What it means for Sentinel
Team formation completed before Phase 1 assessment; no changes after deadline	Purely logistical — lock the roster early. Has no bearing on the architecture, but missing it is a disqualifying error unrelated to build quality.
All ideas, code, documents, and assets must be original and created during the hackathon	The PRD, SRS, architecture blueprint, and the Antigravity master prompt in this document are **planning artifacts** — preparing them ahead of the event is normal and not a violation, the same way an architect brings a plan to a job site. What must happen *during* the hackathon window is the actual codebase: repo creation, commits, and generated UI/backend assets. Do not pre-generate the repository before the event clock starts — have the Antigravity prompt ready to fire the moment it does.
Plagiarism / pre-built projects / unauthorized third-party material → disqualification	Sentinel's core logic (agent pipeline, hash chain, confidence bands) is original design work, not a forked template. The only external material is documented, permitted API integration (AbuseIPDB, VirusTotal, GreyNoise) — cite these openly as data sources, not as borrowed code.
Submissions via Unstop only, within given timelines	No architectural impact — a scheduling/logistics constraint. Build in a buffer before the deadline for the final "no new code" polish window already reflected in Section 6's build order.
Licensed tools/datasets only with valid authorization	AbuseIPDB, VirusTotal, and GreyNoise free tiers are fine to use as-is. Use your own registered API keys — never shared, scraped, or unauthorized credentials — and stay within each service's free-tier terms of use.
All submission links (GitHub, demo, docs) must be public and accessible	Set the repository to public (not "unlisted") before the deadline, and verify the demo video and this document resolve without requiring login — confirm this well before the last hour, not at submission time.
Professional and ethical conduct throughout	No specific architectural implication, but worth noting: Sentinel's own "constrained action space" design principle — the AI can only choose from a pre-approved menu — is a good story to connect back to this rule if asked about responsible AI design in Q&A.
Jury decisions are final; organisers may modify rules/timelines	No action needed — just monitor the Unstop platform for updates through the event in case a rule or timeline shifts.

13. Judging Criteria Alignment

Criterion	Weight	How Sentinel scores against it
Relevance to Problem Statement	—	Directly built for PS7; every agent maps to a specific line in the official challenge statement (compound-signal correlation, behavioral detection, containment orchestration, auditability).
Innovation & Creativity	—	The non-AI Watcher/Record agents (deliberately *not* using an LLM where determinism is more trustworthy) and the bounded retry-then-escalate loop are the differentiators — most competing teams will likely reach for an all-AI pipeline by default.
Technical Implementation	—	Real threat-intel API integration, a genuinely working confidence-gated pipeline, and a functioning (not just claimed) tamper-evident hash chain with a live "Verify Chain" demo moment.
Business Viability	—	Section 9's compliance mapping (NERC CIP, SOC 2, ISO 27001) gives judges a credible hackathon-to-pilot path, not just a demo — pair this with an honest system_auto vs. human_operator action split rather than an inflated "fully autonomous" claim.
Presentation & Clarity	—	The 3-minute demo flow (Section 8) is already structured around a clear, escalating narrative with one explicit "don't skip" proof moment (the ambiguous-case review).
Impact & Scalability	—	The architecture generalizes across IT/OT environments and doesn't require replacing legacy infrastructure — it observes behavior/logs, which is the core scalability argument to make explicit in the pitch close.

Two language precision points to hold under Q&A, since both are exactly where a technically sharp judge will probe:

- Always say **"tamper-evident and verifiable,"** never "immutable" (Section 3.2's framing note already establishes this — keep it consistent everywhere, including live Q&A, not just the written doc).
- State the **actual automation split** demoed (e.g., "X% system_auto, Y% routed to human_operator via the Review Queue") rather than claiming full autonomy — this is more credible on Business Viability than an inflated claim, precisely because it's specific and checkable against what's on screen.